

Spec-Driven Development: Yapay Zekâ Çağında Davranış Tanımı Odaklı Yazılım Geliştirme

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Spec-driven development, yani şartname veya davranış tanımı odaklı geliştirme; yazılım geliştirme sürecinde kod yazmadan önce sistemin nasıl davranması gerektiğini açık biçimde tanımlamaya dayanan bir yaklaşımdır. Bu yaklaşımda asıl amaç, yazılımın yalnızca teknik olarak çalışmasını sağlamak değildir; daha önemli amaç, sistemin beklenen davranışlarını, sınırlarını, hata durumlarını, kullanıcı akışlarını ve kalite beklentilerini önceden netleştirmektir.

Klasik yazılım geliştirme sürecinde geliştirici çoğu zaman önce kod yazar, sonra gerekirse dokümantasyon ekler. Spec-driven development ise bu sırayı tersine çevirir:

Önce sistemin ne yapacağı tanımlanır, sonra bu tanıma uygun kod üretilir.

Bu çalışma, spec-driven development yaklaşımını dört boyutta incelemektedir: klasik yaklaşımla karşılaştırma, spec kavramının içeriği ve nitelikleri, yaklaşımın yapay zekâ destekli geliştirmedeki özel rolü ile doğru ve yanlış uygulama biçimleri. Çalışmanın temel savı şudur: yapay zekâ çağında asıl mesele kodu kimin yazdığı değil, sistemin ne yapması gerektiğinin ne kadar doğru tarif edildiğidir.

Klasik Geliştirme Yaklaşımı ve Sınırları

Klasik geliştirme yaklaşımı çoğu zaman şu sırayla işler:

İhtiyac ortaya çıkar

|

v

Gelistirici kod yazmaya baslar

|

v

Kod calisir hale getirilir

|

v

Testler ve dokumantasyon sonradan eklenir

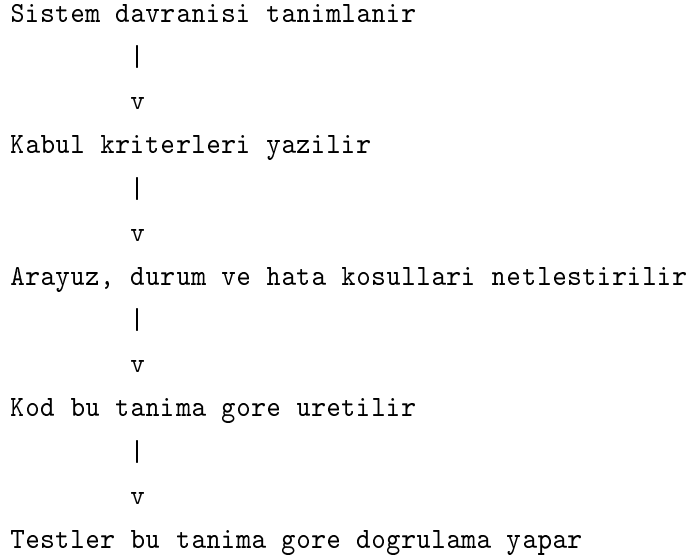
Bu yaklaşım pratik ve hızlı görünebilir; ancak özellikle karmaşık sistemlerde belirli problemler doğurur:

- sistem davranışı baştan net değildir,
- sınır durumları geç fark edilir,
- hata durumları eksik düşünülür,
- iş beklentisi ile kod davranışı ayrışabilir,
- test yazmak sonradan zorlaşır,
- dokümantasyon koddan kopuk hale gelir.

Bu koşullarda kod çalışıyor gibi görünse bile, sistemin gerçekten doğru davranıp davranmadığı belirsiz kalabilir.

Spec-Driven Yaklaşımın Yapısı

Spec-driven development süreci şu şekilde kurar:



Bu yaklaşımda spec, yalnızca açıklayıcı bir doküman değildir; sistemin davranış kontratıdır:

Spec, yazılımın ne yapacağını ve ne yapmayacağını belirleyen davranış haritasıdır.

Spec'in İçeriği

Nitelikli bir spec genellikle şu konuları kapsar:

- özelliğin amacı,
- kullanıcı senaryoları,
- kabul kriterleri,

- girdi ve çıktı yapıları,
- arayüz sözleşmeleri,
- arayüz durumları,
- yükleme durumları,
- hata durumları,
- boş durumlar,
- yetki ve izin kuralları,
- sınır durumları,
- sistemin yapmaması gereken davranışlar,
- test edilmesi gereken durumlar.

Belirsiz ve Nitelikli Tanım Karşılaştırması

Farkı somutlaştırmak için bir giriş ekranı örneği ele alınabilir. Belirsiz bir istek şu biçimdedir:

Giriş sayfası yap.

Bu ifade son derece belirsizdir: yapay zekâ veya insan geliştirici böyle bir istekle bir ekran üretebilir; fakat sistemin doğru davranışlarını eksik bırakabilir. Nitelikli bir spec ise şöyle görünür:

Giriş ekranı e-posta ve şifre alanlarından oluşmalıdır. E-posta boş bırakılırsa kullanıcıya hata mesajı gösterilmelidir. Şifre alanı boşsa form gönderilmemelidir. Gönderim sırasında düğme devre dışı kalmalı ve yükleme durumu gösterilmelidir. Sunucudan yetkisiz erişim yanıtı dönerse kullanıcıya geçersiz giriş bilgisi mesajı gösterilmelidir. Başarılı girişten sonra kullanıcı ana panele yönlendirilmelidir.

İkinci tanım yalnızca bir ekran istemez; sistem davranışını tarif eder. Belirsiz tanımın açık bıraktığı sorular — hangi alanlar olacak, doğrulama nasıl yapılacak, hata mesajları ne olacak, yükleme durumu gösterilecek mi, başarılı ve başarısız girişte ne olacak, kullanıcı zaten giriş yapmışsa ne olacak — nitelikli tanımda önceden yanıtlanmıştır.

Neden Sadece Kod Yazmak Yetmez?

Kod, sistemin görünen gerçekleştirimidir. Ancak koddan önce yanıtlanması gereken daha temel sorular vardır:

- Bu özellik tam olarak ne yapacak?
- Hangi kullanıcı bunu kullanacak?
- Hangi durumda başarılı sayılacak?

- Hangi durumda hata verecek?
- Kullanıcı yanlış veri girerse ne olacak?
- Sistem yavaş yanıt verirse arayüz nasıl davranacak?
- Kullanıcının yetkisi yoksa ne gösterilecek?
- Veri yoksa ekran nasıl görünecek?
- Dış sistem başarısız olursa sistem nasıl toparlanacak?

Bu sorular yanıtlanmadan kod yazıldığında, yazılım hızlı üretilmiş gibi görünür; fakat çoğu zaman sistem davranışı eksik kalır. Spec-driven development'ın temel iddiası budur:

Kod yazmadan önce davranış netleştirilmelidir.

Spec ile Klasik Dokümantasyon Arasındaki Fark

Spec-driven development, klasik anlamda uzun ve statik dokümantasyon yazmak anlamına gelmez. Klasik dokümantasyon çoğu zaman uzun açıklamalar içerir, koddan kopuk kalabilir, zamanla güncelliğini kaybeder ve geliştiriciler tarafından nadiren okunur.

Nitelikli bir spec ise farklıdır:

- kısa ve nettir,
- davranış odaklıdır,
- test edilebilir,
- yapay zekâya doğrudan verilebilir,
- kabul kriteri olarak kullanılabilir,
- kodla birlikte güncellenebilir,
- sistemin doğruluğunu ölçmeye yarar.

Bu nedenle spec, ölü bir doküman değil, yaşayan bir davranış kontratı olmalıdır.

Spec ve Test İlişkisi

Spec-driven development doğal olarak test odaklı düşünmeyi destekler; çünkü nitelikli bir spec, test edilebilir ifadelerden oluşur. Örneğin “sunucudan yetkisiz erişim yanıtı dönerse kullanıcıya geçersiz giriş bilgisi mesajı gösterilmelidir” biçimindeki bir spec maddesi, doğrudan bir davranış testine dönüştürülebilir: kullanıcı yanlış giriş bilgisi gönderdiğinde ve sunucu yetkisiz erişim yanıtı döndürdüğünde, ekranda geçersiz giriş bilgisi mesajı görünmelidir.

Bu dönüşüm zinciri şu şekilde gösterilebilir:

Spec maddesi

|

v

Kabul kriteri

|

v

Test senaryosu

|

v

Gerçekleştirim doğrulaması

Böylece spec, testlerin kaynağı haline gelir: her spec maddesi bir kabul kriterine, her kabul kriteri bir test senaryosuna karşılık gelir ve gerileme hataları bu zincir üzerinden yakalanabilir.

Yapay Zekâ Çağında Spec-Driven Development

Yapay zekâ destekli yazılım geliştirmede spec-driven development daha da önemli hale gelmiştir; çünkü yapay zekâ, verilen talimata göre kod üretir. Talimat belirsizse, üretilen kod da belirsiz, eksik veya yanlış olabilir. Yapay zekâ için spec şu işlevi görür:

Spec, yapay zekânın kod üretirken izleyeceği davranış mimarisidir.

Yapay zekâ destekli süreçte spec, şu zincirin başlangıcı olabilir:

Spec

|

v

Yapay zeka ile kod üretimi

|

v

Test üretimi

|

v

Kod incelemesi

|

v

Davranış doğrulaması

Yani iyi yazılmış bir spec yalnızca kod üretmek için kullanılmaz; aynı zamanda test üretmek, kabul kriterleri belirlemek ve gerileme hatalarını yakalamak için de kullanılabilir. Bu çok işlevlilik şu şekilde özetlenebilir:

Spec; yapay zekâ için talimat, geliştirici için kontrol listesi, test sistemi için doğrulama kriteridir.

Spec ve Davranış Geometrisi

Spec-driven development, bir sistemin davranış geometrisini önceden tarif etme çabası olarak da görülebilir. Burada davranış geometrisi şu anlama gelir:

Sistemin hangi durumda hangi yöne akacağını, hangi sınırlar içinde çalışacağını ve hangi durumlarda nasıl tepki vereceğini belirleyen yapı.

Bir sistemin davranış geometrisi net değilse, kod yazmak yalnızca belirsizliği hızlandırır: yapay zekâ bu durumda daha hızlı üretim yapar, fakat ürettiği şey daha hızlı kaos da olabilir. Bu nedenle yapay zekâ çağında şu ilke belirleyicidir:

Yapay zekâ kodu hızlı yazabilir; fakat sistemin davranış geometrisini insan tarif etmelidir.

Geliştiricinin Rolündeki Dönüşüm

Yapay zekâ öncesi dönemde geliştiricinin temel değeri çoğu zaman kod yazma becerisiyle ölçülüyordu. Yapay zekâ destekli geliştirmede ise geliştiricinin değeri şu alanlara kaymaktadır:

- doğru problemi tanımlamak,
- sistem sınırlarını belirlemek,
- davranışları açık ifade etmek,
- yapay zekânın ürettiği kodu denetlemek,
- mimari kaliteyi korumak,
- test edilebilirlik sağlamak,
- sınır durumlarını görmek,
- ürün hedefi ile teknik çözümü hizalamak.

Bu dönüşüm geliştiricinin önemsizleşmesi anlamına gelmez; aksine geliştiricinin rolü daha soyut, daha mimari ve daha karar odaklı hale gelir. Geliştirici, yalnızca gerçekleştirim yapan kişi olmaktan çıkar; sistemin davranışını tasarlayan ve doğruluğunu denetleyen kişiye dönüşür.

Yaklaşımın Tartışmalı Yönü

Spec-driven development, özellikle yapay zekâ çağında tartışmalıdır; çünkü birçok geliştirici için kod yazmak işin en keyifli kısmıdır. Spec yazmak ise sıkıcı, yavaşlatıcı ve bürokratik görünebilir. Bu itiraz şu şekilde özetlenebilir:

Neden işin eğlenceli kısmı olan kodlamayı yapay zekâya verip, geliştiriciye doküman yazmayı bırakıyoruz?

Bu itiraz haklı bir taraf taşır: eğer spec-driven development yalnızca geliştiriciye daha fazla doküman yazdırmak anlamına gelirse, yaklaşım verimsiz ve motivasyon kırıcı hale gelir. Ancak doğru uygulandığında spec yazmak, klasik doküman yazmak değildir; sistemin davranışını netleştiren kısa, test edilebilir ve üretime bağlı bir yapı kurmaktır.

Doğru ve Yanlış Uygulama Karşılaştırması

Yanlış Uygulama

Yaklaşımın kötü uygulanması şu görünümü alır:

- uzun ve okunmayan dokümanlar yazılır,
- spec koddan ayrı yaşar,
- geliştirici yalnızca dokümantasyon işçisi gibi kullanılır,
- yapay zekâ kontrolsüz kod üretir,
- testler spec ile bağlantılı değildir,
- doküman güncel kalmaz.

Bu uygulama biçimi ek riskler de üretir: süreç gereksiz bürokratikleşir, gerçek ihtiyaçlar yerine kâğıt üzerindeki tanımlar takip edilir ve yaratıcılık ile hızlı deneme kültürü zayıflar.

Doğru Uygulama

Doğru uygulama ise şu ilkelere dayanır:

- spec kısa ve davranış odaklıdır,
- her spec maddesi test edilebilir,
- yapay zekâ kod üretirken spec'i kullanır,
- testler spec'ten türetilir,
- geliştirici kodu ve davranışı denetler,
- spec sistemin yaşayan kontratı haline gelir.

İki uygulama biçimi arasındaki ayırt edici ölçüt şudur: spec, üretim sürecinin dışında duran bir rapor mudur, yoksa üretim sürecinin içinde işleyen bir kontrat mıdır? Birinci durumda yaklaşım yüke, ikinci durumda kaldıraca dönüşür.

Sonuç

Spec-driven development, doğru anlaşıldığında yazılım geliştirmeyi yavaşlatan bir dokümantasyon yükü değildir; aksine, sistemin ne yapacağını netleştirerek hem insan geliştiricinin hem de yapay zekâ aracının daha doğru üretim yapmasını sağlar. Yaklaşım şu şekilde tanımlanabilir:

Spec-driven development, kod yazmadan önce sistemin davranışını, sınırlarını, hata durumlarını ve kabul kriterlerini tanımlama yaklaşımıdır.

Yapay zekâ çağındaki daha özel tanımı ise şudur:

Yapay zekâ destekli spec-driven development; insanın sistem davranışını tarif ettiği, yapay zekânın bu tarife göre kod ürettiği ve geliştiricinin üretilen kodu kalite, mimari ve doğruluk açısından denetlediği geliştirme yaklaşımıdır.

Yaklaşımın başarısı, spec'in uzun, soyut ve güncellenmeyen bir doküman olmamasına bağlıdır: spec kısa, davranış odaklı, test edilebilir ve kodla birlikte yaşayan bir yapı olmalıdır. Sonuç olarak yapay zekâ çağında asıl mesele kodu kimin yazdığı değildir; asıl mesele, sistemin ne yapması gerektiğinin ne kadar doğru tarif edildiğidir:

Yapay zekâ gerçekleştirmeyi hızlandırabilir; fakat doğru sistemi tarif etmek hâlâ insanın sorumluluğundadır.